

UCM Competitive Programming Handbook

Programación competitiva — Universidad Católica del Maule

UCM Competitive Programming

Índice

UCM Competitive Programming Handbook	4
Preface	5
Créditos de implementaciones UTFSM	6
Part I: Basic techniques	7
1 Introduction	8
1.1 Programming languages	8
1.2 Input and output	8
1.3 Working with numbers	8
1.4 Shortening code	8
1.5 Mathematics	8
1.6 Contests and resources	8
2 Time complexity	9
2.1 Calculation rules	9
2.2 Complexity classes	9
2.3 Estimating efficiency	9
2.4 Maximum subarray sum	9
3 Sorting	10
3.1 Sorting theory	10
3.2 Sorting in C++	10
3.3 Binary search	10
4 Data structures	11
4.1 Dynamic arrays	11
4.2 Set structures	11
4.3 Map structures	11
4.4 Iterators and ranges	11
4.5 Other structures	11
4.6 Comparison to sorting	11
5 Complete search	12
5.1 Generating subsets	12
5.2 Generating permutations	12
5.3 Backtracking	12
5.4 Pruning the search	12
5.5 Meet in the middle	12
6 Greedy algorithms	13
6.1 Coin problem	13
6.2 Scheduling	13
6.3 Tasks and deadlines	13
6.4 Minimizing sums	13
6.5 Data compression	13
7 Dynamic programming	14
7.1 Coin problem	14
7.2 Longest increasing subsequence	14
7.3 Paths in a grid	14
7.4 Knapsack problems	14
7.5 Edit distance	14

7.6 Counting tilings	14
Complemento UTFSM: optimización de programación dinámica	14
Li Chao Tree	14
8 Amortized analysis	16
8.1 Two pointers method	16
8.2 Nearest smaller elements	16
8.3 Sliding window minimum	16
9 Range queries	17
9.1 Static array queries	17
9.2 Binary indexed tree	17
9.3 Segment tree	17
9.4 Additional techniques	17
Complemento UTFSM: estructuras avanzadas de rango	17
Union-find con rollback	17
Segment tree lazy	18
Segment tree persistente	19
Errores frecuentes	21
10 Bit manipulation	22
10.1 Bit representation	22
10.2 Bit operations	22
10.3 Representing sets	22
10.4 Bit optimizations	22
10.5 Dynamic programming	22
Part II: Graph algorithms	23
11 Basics of graphs	24
11.1 Graph terminology	24
11.2 Graph representation	24
12 Graph traversal	25
12.1 Depth-first search	25
12.2 Breadth-first search	25
12.3 Applications	25
13 Shortest paths	26
13.1 Bellman–Ford algorithm	26
13.2 Dijkstra’s algorithm	26
13.3 Floyd–Warshall algorithm	26
14 Tree algorithms	27
14.1 Tree traversal	27
14.2 Diameter	27
14.3 All longest paths	27
14.4 Binary trees	27
15 Spanning trees	28
15.1 Kruskal’s algorithm	28
15.2 Union-find structure	28
15.3 Prim’s algorithm	28
16 Directed graphs	29
16.1 Topological sorting	29
16.2 Dynamic programming	29
16.3 Successor paths	29
16.4 Cycle detection	29
17 Strong connectivity	30
17.1 Kosaraju’s algorithm	30
17.2 2SAT problem	30
18 Tree queries	31
18.1 Finding ancestors	31
18.2 Subtrees and paths	31
18.3 Lowest common ancestor	31
18.4 Offline algorithms	31
Complemento UTFSM: consultas avanzadas en árboles	31
Heavy-Light Decomposition	31
Uso seguro	32
19 Paths and circuits	33

19.1 Eulerian paths	33
19.2 Hamiltonian paths	33
19.3 De Bruijn sequences	33
19.4 Knight's tours	33
20 Flows and cuts	34
20.1 Ford–Fulkerson algorithm	34
20.2 Disjoint paths	34
20.3 Maximum matchings	34
20.4 Path covers	34
Complemento UTFSM: flujos y matching	34
Dinic	34
Hopcroft-Karp	35
Checklist	37

Part III: Advanced topics 38

21 Number theory	39
21.1 Primes and factors	39
21.2 Modular arithmetic	39
21.3 Solving equations	39
21.4 Other results	39
Complemento UTFSM: teoría de números avanzada	39
Teorema chino del resto	39
Miller-Rabin determinista para 64 bits	40
Precauciones	41
22 Combinatorics	42
22.1 Binomial coefficients	42
22.2 Catalan numbers	42
22.3 Inclusion-exclusion	42
22.4 Burnside's lemma	42
22.5 Cayley's formula	42
23 Matrices	43
23.1 Operations	43
23.2 Linear recurrences	43
23.3 Graphs and matrices	43
Complemento UTFSM: multiplicación rápida de polinomios	43
Number Theoretic Transform	43
24 Probability	45
24.1 Calculation	45
24.2 Events	45
24.3 Random variables	45
24.4 Markov chains	45
24.5 Randomized algorithms	45
25 Game theory	46
25.1 Game states	46
25.2 Nim game	46
25.3 Sprague–Grundy theorem	46
26 String algorithms	47
26.1 String terminology	47
26.2 Trie structure	47
26.3 String hashing	47
26.4 Z-algorithm	47
Complemento UTFSM: algoritmos avanzados de strings	47
KMP	47
Aho-Corasick	47
Suffix array y LCP	49
Manacher	50
27 Square root algorithms	52
27.1 Combining algorithms	52
27.2 Integer partitions	52
27.3 Mo's algorithm	52
28 Segment trees revisited	53
28.1 Lazy propagation	53

28.2	Dynamic trees	53
28.3	Data structures	53
28.4	Two-dimensionality	53
29	Geometry	54
29.1	Complex numbers	54
29.2	Points and lines	54
29.3	Polygon area	54
29.4	Distance functions	54
	Complemento UTFSM: geometría computacional avanzada	54
	Convex hull	54
	Punto en polígono	55
	Intersección de semiplanos	55
	Checklist numérico	57
30	Sweep line algorithms	58
30.1	Intersection points	58
30.2	Closest pair problem	58
30.3	Convex hull problem	58

Bibliography	59
---------------------	-----------

UCM Competitive Programming Handbook

Preface

Créditos de implementaciones UTFSM

Los complementos marcados como UTFSM parten de implementaciones del Handbook de Programación Competitiva UTFSM. Cada snippet indica junto al código a la persona acreditada por la implementación de referencia.

La tabla completa, con el archivo UCM y su archivo de origen, está disponible en [snippets/ATTRIBUTION.md](#). El material de referencia se distribuye bajo licencia MIT.

Part I: Basic techniques

1 Introduction

1.1 Programming languages

1.2 Input and output

1.3 Working with numbers

1.4 Shortening code

1.5 Mathematics

1.6 Contests and resources

2 Time complexity

2.1 Calculation rules

2.2 Complexity classes

2.3 Estimating efficiency

2.4 Maximum subarray sum

3 Sorting

3.1 Sorting theory

3.2 Sorting in C++

3.3 Binary search

4 Data structures

4.1 Dynamic arrays

4.2 Set structures

4.3 Map structures

4.4 Iterators and ranges

4.5 Other structures

4.6 Comparison to sorting

5 Complete search

5.1 Generating subsets

5.2 Generating permutations

5.3 Backtracking

5.4 Pruning the search

5.5 Meet in the middle

6 Greedy algorithms

6.1 Coin problem

6.2 Scheduling

6.3 Tasks and deadlines

6.4 Minimizing sums

6.5 Data compression

7 Dynamic programming

7.1 Coin problem

7.2 Longest increasing subsequence

7.3 Paths in a grid

7.4 Knapsack problems

7.5 Edit distance

7.6 Counting tilings

Complemento UTFSM: optimización de programación dinámica

Li Chao Tree

Mantiene el mínimo de rectas $m \cdot x + b$ sobre un dominio entero fijo y cerrado. Insertar una recta y consultar un punto cuestan $O(\log C)$, donde C es el tamaño del intervalo de coordenadas. Declara el dominio al construirlo y verifica que no haya overflow en $m \cdot x + b$.

Crédito UTFSM: Carlos Lagos contribuyó la implementación de referencia en el Handbook UTFSM.

```
#include <algorithm>
#include <limits>
#include <vector>

class LiChaoTree {
public:
    LiChaoTree(long long left, long long right)
        : left_(left), right_(right), tree_(4) {}

    void add_line(long long slope, long long intercept) {
        add_line(Line{slope, intercept}, 1, left_, right_);
    }

    long long minimum(long long coordinate) const {
        return minimum(coordinate, 1, left_, right_);
    }

private:
    struct Line {
        long long slope = 0;
        long long intercept = std::numeric_limits<long long>::max() / 4;

        long long value(long long coordinate) const {
            return slope * coordinate + intercept;
        }
    };

    void add_line(Line line, int node, long long left, long long right) {
        if (node >= static_cast<int>(tree_.size())) {
            tree_.resize(node + 1);
        }
        const long long middle = left + (right - left) / 2;
        bool better_left = line.value(left) < tree_[node].value(left);
        const bool better_middle = line.value(middle) < tree_[node].value(middle);
        if (better_middle) {
            std::swap(line, tree_[node]);
        }
        if (left == right) {
            return;
        }
        if (better_left != better_middle) {
            add_line(line, node * 2, left, middle);
        } else {
            add_line(line, node * 2 + 1, middle, right);
        }
    }

    long long left_, right_;
    std::vector<Line> tree_;
};
```

```

        add_line(line, node * 2 + 1, middle + 1, right);
    }
}

long long minimum(long long coordinate, int node, long long left,
                  long long right) const {
    if (node >= static_cast<int>(tree_.size())) {
        return std::numeric_limits<long long>::max() / 4;
    }
    const long long current = tree_[node].value(coordinate);
    if (left == right) {
        return current;
    }
    const long long middle = left + (right - left) / 2;
    if (coordinate <= middle) {
        return std::min(current, minimum(coordinate, node * 2, left, middle));
    }
    return std::min(current, minimum(coordinate, node * 2 + 1, middle + 1, right));
}

long long left_;
long long right_;
std::vector<Line> tree_;
};

```

Es apropiado para transiciones DP que minimizan expresiones lineales; para máximos, niega pendientes e interceptos, y vuelve a negar la respuesta.

8 Amortized analysis

8.1 Two pointers method

8.2 Nearest smaller elements

8.3 Sliding window minimum

9 Range queries

9.1 Static array queries

9.2 Binary indexed tree

9.3 Segment tree

9.4 Additional techniques

Complemento UTFSM: estructuras avanzadas de rango

Union-find con rollback

Mantiene componentes conexas reversibles, útil para conectividad offline con árbol de segmentos sobre el tiempo. `unite` cuesta $O(\log V)$ y `rollback(snapshot)` revierte las operaciones posteriores. No usa compresión de caminos para conservar el rollback.

Crédito UTFSM: Sebastián Torrealba contribuyó la implementación de referencia en el Handbook UTFSM.

```
#include <numeric>
#include <utility>
#include <vector>

class RollbackDisjointSet {
public:
    explicit RollbackDisjointSet(int element_count)
        : parent_(element_count), size_(element_count, 1), components_(element_count) {
        std::iota(parent_.begin(), parent_.end(), 0);
    }

    int find(int element) const {
        while (parent_[element] != element) {
            element = parent_[element];
        }
        return element;
    }

    bool unite(int first, int second) {
        first = find(first);
        second = find(second);
        if (first == second) {
            history_.push_back({-1, -1});
            return false;
        }
        if (size_[first] < size_[second]) {
            std::swap(first, second);
        }
        history_.push_back({second, size_[first]});
        parent_[second] = first;
        size_[first] += size_[second];
        --components_;
        return true;
    }

    int snapshot() const { return static_cast<int>(history_.size()); }

    void rollback(int snapshot) {
        while (static_cast<int>(history_.size()) > snapshot) {
            const auto [merged_root, old_size] = history_.back();
            history_.pop_back();
            if (merged_root == -1) {
                continue;
            }
            const int new_root = parent_[merged_root];
```

```

        parent_[merged_root] = merged_root;
        size_[new_root] = old_size;
        ++components_;
    }
}

int component_count() const { return components_; }

private:
    std::vector<int> parent_;
    std::vector<int> size_;
    std::vector<std::pair<int, int>> history_;
    int components_;
};

```

Segment tree lazy

Esta versión implementa suma de rango y suma de valores sobre un intervalo cerrado [left, right]. Tanto add como sum cuestan $O(\log N)$ y el árbol usa $O(N)$ memoria. El vector inicial debe ser no vacío.

Crédito UTFSM: Gabriel Carmona contribuyó la implementación de referencia en el Handbook UTFSM.

```

#include <algorithm>
#include <vector>

class LazySegmentTree {
public:
    explicit LazySegmentTree(const std::vector<long long>& values)
        : size_(static_cast<int>(values.size())), tree_(4 * size_), lazy_(4 * size_) {
        build(1, 0, size_ - 1, values);
    }

    void add(int left, int right, long long value) {
        add(1, 0, size_ - 1, left, right, value);
    }

    long long sum(int left, int right) {
        return sum(1, 0, size_ - 1, left, right);
    }

private:
    void build(int node, int left, int right, const std::vector<long long>& values) {
        if (left == right) {
            tree_[node] = values[left];
            return;
        }
        const int middle = left + (right - left) / 2;
        build(node * 2, left, middle, values);
        build(node * 2 + 1, middle + 1, right, values);
        tree_[node] = tree_[node * 2] + tree_[node * 2 + 1];
    }

    void apply(int node, int left, int right, long long value) {
        tree_[node] += value * (right - left + 1);
        lazy_[node] += value;
    }

    void push(int node, int left, int right) {
        if (lazy_[node] == 0 || left == right) {
            return;
        }
    }
}

```

```

    const int middle = left + (right - left) / 2;
    apply(node * 2, left, middle, lazy_[node]);
    apply(node * 2 + 1, middle + 1, right, lazy_[node]);
    lazy_[node] = 0;
}

void add(int node, int left, int right, int query_left, int query_right,
        long long value) {
    if (query_right < left || right < query_left) {
        return;
    }
    if (query_left <= left && right <= query_right) {
        apply(node, left, right, value);
        return;
    }
    push(node, left, right);
    const int middle = left + (right - left) / 2;
    add(node * 2, left, middle, query_left, query_right, value);
    add(node * 2 + 1, middle + 1, right, query_left, query_right, value);
    tree_[node] = tree_[node * 2] + tree_[node * 2 + 1];
}

long long sum(int node, int left, int right, int query_left, int query_right) {
    if (query_right < left || right < query_left) {
        return 0;
    }
    if (query_left <= left && right <= query_right) {
        return tree_[node];
    }
    push(node, left, right);
    const int middle = left + (right - left) / 2;
    return sum(node * 2, left, middle, query_left, query_right) +
           sum(node * 2 + 1, middle + 1, right, query_left, query_right);
}

int size_;
std::vector<long long> tree_;
std::vector<long long> lazy_;
};

```

Segment tree persistente

Cada actualización puntual genera una versión nueva. `update` devuelve su identificador y `sum(version, left, right)` consulta cualquier versión en $O(\log N)$; la memoria total es $O(N + Q \log N)$.

Crédito UTFSM: Sebastián Torrealba contribuyó la implementación de referencia en el Handbook UTFSM.

```

#include <vector>

class PersistentSegmentTree {
public:
    explicit PersistentSegmentTree(const std::vector<long long>& values)
        : size_(static_cast<int>(values.size())) {
        roots_.push_back(build(0, size_ - 1, values));
    }

    int update(int version, int position, long long value) {
        roots_.push_back(update(roots_[version], 0, size_ - 1, position, value));
        return static_cast<int>(roots_.size()) - 1;
    }
}

```

```

long long sum(int version, int left, int right) const {
    return sum(roots_[version], 0, size_ - 1, left, right);
}

private:
struct Node {
    long long sum;
    int left_child;
    int right_child;
};

int make_node(long long sum, int left_child = -1, int right_child = -1) {
    nodes_.push_back({sum, left_child, right_child});
    return static_cast<int>(nodes_.size()) - 1;
}

int build(int left, int right, const std::vector<long long>& values) {
    if (left == right) {
        return make_node(values[left]);
    }
    const int middle = left + (right - left) / 2;
    const int left_child = build(left, middle, values);
    const int right_child = build(middle + 1, right, values);
    return make_node(nodes_[left_child].sum + nodes_[right_child].sum,
        left_child, right_child);
}

int update(int node, int left, int right, int position, long long value) {
    if (left == right) {
        return make_node(value);
    }
    const int middle = left + (right - left) / 2;
    int left_child = nodes_[node].left_child;
    int right_child = nodes_[node].right_child;
    if (position <= middle) {
        left_child = update(left_child, left, middle, position, value);
    } else {
        right_child = update(right_child, middle + 1, right, position, value);
    }
    return make_node(nodes_[left_child].sum + nodes_[right_child].sum,
        left_child, right_child);
}

long long sum(int node, int left, int right, int query_left,
    int query_right) const {
    if (query_right < left || right < query_left) {
        return 0;
    }
    if (query_left <= left && right <= query_right) {
        return nodes_[node].sum;
    }
    const int middle = left + (right - left) / 2;
    return sum(nodes_[node].left_child, left, middle, query_left, query_right) +
        sum(nodes_[node].right_child, middle + 1, right, query_left,
            query_right);
}

int size_;
std::vector<Node> nodes_;
std::vector<int> roots_;
};

```

Errores frecuentes

- No mezcles rangos abiertos con estos intervalos cerrados.
- Conserva el número de versión retornado por **update**.
- Toma un **snapshot** antes del bloque temporal que deseas deshacer.

10 Bit manipulation

10.1 Bit representation

10.2 Bit operations

10.3 Representing sets

10.4 Bit optimizations

10.5 Dynamic programming

Part II: Graph algorithms

11 Basics of graphs

11.1 Graph terminology

11.2 Graph representation

12 Graph traversal

12.1 Depth-first search

12.2 Breadth-first search

12.3 Applications

13 Shortest paths

13.1 Bellman–Ford algorithm

13.2 Dijkstra’s algorithm

13.3 Floyd–Warshall algorithm

14 Tree algorithms

14.1 Tree traversal

14.2 Diameter

14.3 All longest paths

14.4 Binary trees

15 Spanning trees

15.1 Kruskal's algorithm

15.2 Union-find structure

15.3 Prim's algorithm

16 Directed graphs

16.1 Topological sorting

16.2 Dynamic programming

16.3 Successor paths

16.4 Cycle detection

17 Strong connectivity

17.1 Kosaraju's algorithm

17.2 2SAT problem

18 Tree queries

18.1 Finding ancestors

18.2 Subtrees and paths

18.3 Lowest common ancestor

18.4 Offline algorithms

Complemento UTFSM: consultas avanzadas en árboles

Heavy-Light Decomposition

Descompone un árbol en segmentos contiguos para delegar consultas de camino a una estructura de rangos. La construcción toma $O(V)$ y un camino se expresa en $O(\log V)$ segmentos. `path_segments(u, v)` devuelve intervalos inclusivos; para valores en aristas usa `edge_values = true`. El orden de los segmentos no sirve directamente para operaciones no conmutativas.

Crédito UTFSM: Javier Oliva figura como autor de la implementación de referencia en el Handbook UTFSM.

```
#include <algorithm>
#include <utility>
#include <vector>

class HeavyLightDecomposition {
public:
    HeavyLightDecomposition(const std::vector<std::vector<int>>& adjacency, int root = 0)
        : adjacency_(adjacency), parent_(adjacency.size(), -1),
          depth_(adjacency.size()), subtree_size_(adjacency.size()),
          heavy_child_(adjacency.size(), -1), head_(adjacency.size()),
          position_(adjacency.size()) {
        compute_subtrees(root);
        int next_position = 0;
        decompose(root, root, next_position);
    }

    int position(int vertex) const { return position_[vertex]; }

    std::vector<std::pair<int, int>> path_segments(int first, int second,
                                                  bool edge_values = false) const {
        std::vector<std::pair<int, int>> segments;
        while (head_[first] != head_[second]) {
            if (depth_[head_[first]] < depth_[head_[second]]) {
                std::swap(first, second);
            }
            segments.push_back({position_[head_[first]], position_[first]});
            first = parent_[head_[first]];
        }
        if (depth_[first] < depth_[second]) {
            std::swap(first, second);
        }
        const int start = position_[second] + (edge_values ? 1 : 0);
        if (start <= position_[first]) {
            segments.push_back({start, position_[first]});
        }
        return segments;
    }

    std::pair<int, int> subtree_segment(int vertex) const {
        return {position_[vertex], position_[vertex] + subtree_size_[vertex] - 1};
    }

private:
    int compute_subtrees(int vertex) {
```

```

subtree_size_[vertex] = 1;
int largest_child_size = 0;
for (const int neighbour : adjacency_[vertex]) {
    if (neighbour == parent_[vertex]) {
        continue;
    }
    parent_[neighbour] = vertex;
    depth_[neighbour] = depth_[vertex] + 1;
    const int child_size = compute_subtrees(neighbour);
    subtree_size_[vertex] += child_size;
    if (child_size > largest_child_size) {
        largest_child_size = child_size;
        heavy_child_[vertex] = neighbour;
    }
}
return subtree_size_[vertex];
}

void decompose(int vertex, int chain_head, int& next_position) {
    head_[vertex] = chain_head;
    position_[vertex] = next_position++;
    if (heavy_child_[vertex] != -1) {
        decompose(heavy_child_[vertex], chain_head, next_position);
    }
    for (const int neighbour : adjacency_[vertex]) {
        if (neighbour != parent_[vertex] && neighbour != heavy_child_[vertex]) {
            decompose(neighbour, neighbour, next_position);
        }
    }
}

const std::vector<std::vector<int>>& adjacency_;
std::vector<int> parent_;
std::vector<int> depth_;
std::vector<int> subtree_size_;
std::vector<int> heavy_child_;
std::vector<int> head_;
std::vector<int> position_;
};

```

Uso seguro

- El grafo debe ser un árbol conexo y sus vértices deben estar en $[0, V)$.
- Construye luego un segment tree o Fenwick sobre `position(v)`.
- Usa `subtree_segment(v)` para consultas sobre el subárbol de v .

19 Paths and circuits

19.1 Eulerian paths

19.2 Hamiltonian paths

19.3 De Bruijn sequences

19.4 Knight's tours

20 Flows and cuts

20.1 Ford–Fulkerson algorithm

20.2 Disjoint paths

20.3 Maximum matchings

20.4 Path covers

Complemento UTFSM: flujos y matching

Estas implementaciones complementan el capítulo con soluciones listas para integrar en C++17. Se revisaron a partir de material del Handbook UTFSM; la procedencia detallada y la licencia se indican en la sección de créditos.

Dinic

Calcula flujo máximo en una red dirigida con capacidades enteras no negativas. Su complejidad es $O(V^2 E)$ en el peor caso y usa $O(V + E)$ memoria. Crea el grafo con vértices en $[0, V)$ y llama a `max_flow(source, sink)`; no reutilices la misma instancia si necesitas conservar las capacidades originales.

Crédito UTFSM: Pablo Messina figura como autor de la implementación de referencia en el Handbook UTFSM.

```
#include <algorithm>
#include <limits>
#include <queue>
#include <vector>

class Dinic {
public:
    struct Edge {
        int to;
        int reverse_index;
        long long capacity;
    };

    explicit Dinic(int vertex_count)
        : graph_(vertex_count), level_(vertex_count), next_edge_(vertex_count) {}

    void add_edge(int from, int to, long long capacity) {
        Edge forward{to, static_cast<int>(graph_[to].size()), capacity};
        Edge backward{from, static_cast<int>(graph_[from].size()), 0};
        graph_[from].push_back(forward);
        graph_[to].push_back(backward);
    }

    long long max_flow(int source, int sink) {
        long long flow = 0;
        while (build_levels(source, sink)) {
            std::fill(next_edge_.begin(), next_edge_.end(), 0);
            while (long long pushed = send_flow(source, sink,
                                                std::numeric_limits<long long>::max())) {
                flow += pushed;
            }
        }
        return flow;
    }

private:
    bool build_levels(int source, int sink) {
        std::fill(level_.begin(), level_.end(), -1);
        std::queue<int> pending;
        level_[source] = 0;
        pending.push(source);
    }
};
```

```

while (!pending.empty()) {
    const int vertex = pending.front();
    pending.pop();
    for (const Edge& edge : graph_[vertex]) {
        if (edge.capacity > 0 && level_[edge.to] == -1) {
            level_[edge.to] = level_[vertex] + 1;
            pending.push(edge.to);
        }
    }
}
return level_[sink] != -1;
}

long long send_flow(int vertex, int sink, long long available) {
    if (vertex == sink) {
        return available;
    }
    for (int& edge_index = next_edge_[vertex];
         edge_index < static_cast<int>(graph_[vertex].size()); ++edge_index) {
        Edge& edge = graph_[vertex][edge_index];
        if (edge.capacity == 0 || level_[edge.to] != level_[vertex] + 1) {
            continue;
        }
        const long long pushed =
            send_flow(edge.to, sink, std::min(available, edge.capacity));
        if (pushed == 0) {
            continue;
        }
        edge.capacity -= pushed;
        graph_[edge.to][edge.reverse_index].capacity += pushed;
        return pushed;
    }
    return 0;
}

std::vector<std::vector<Edge>> graph_;
std::vector<int> level_;
std::vector<int> next_edge_;
};

```

Hopcroft-Karp

Encuentra un matching máximo bipartito en $O(E \sqrt{V})$. Los vértices de ambos lados se indexan desde cero y `left_matching()` [u] devuelve el vértice derecho emparejado con u, o -1.

Crédito UTFSM: Abner Vidal contribuyó la implementación de referencia en el Handbook UTFSM.

```

#include <algorithm>
#include <queue>
#include <vector>

class HopcroftKarp {
public:
    HopcroftKarp(int left_size, int right_size)
        : adjacency_(left_size), left_match_(left_size, -1),
          right_match_(right_size, -1), distance_(left_size) {}

    void add_edge(int left_vertex, int right_vertex) {
        adjacency_[left_vertex].push_back(right_vertex);
    }

    int maximum_matching() {

```

```

int matching_size = 0;
while (build_layers()) {
    for (int left_vertex = 0;
         left_vertex < static_cast<int>(adjacency_.size()); ++left_vertex) {
        if (left_match_[left_vertex] == -1 && augment(left_vertex)) {
            ++matching_size;
        }
    }
}
return matching_size;
}

const std::vector<int>& left_matching() const { return left_match_; }

private:
bool build_layers() {
    std::queue<int> pending;
    std::fill(distance_.begin(), distance_.end(), -1);
    for (int left_vertex = 0;
         left_vertex < static_cast<int>(adjacency_.size()); ++left_vertex) {
        if (left_match_[left_vertex] == -1) {
            distance_[left_vertex] = 0;
            pending.push(left_vertex);
        }
    }

    bool found_augmenting_path = false;
    while (!pending.empty()) {
        const int left_vertex = pending.front();
        pending.pop();
        for (const int right_vertex : adjacency_[left_vertex]) {
            const int matched_left = right_match_[right_vertex];
            if (matched_left == -1) {
                found_augmenting_path = true;
            } else if (distance_[matched_left] == -1) {
                distance_[matched_left] = distance_[left_vertex] + 1;
                pending.push(matched_left);
            }
        }
    }
    return found_augmenting_path;
}

bool augment(int left_vertex) {
    for (const int right_vertex : adjacency_[left_vertex]) {
        const int matched_left = right_match_[right_vertex];
        if (matched_left == -1 ||
            (distance_[matched_left] == distance_[left_vertex] + 1 &&
             augment(matched_left))) {
            left_match_[left_vertex] = right_vertex;
            right_match_[right_vertex] = left_vertex;
            return true;
        }
    }
    distance_[left_vertex] = -1;
    return false;
}

std::vector<std::vector<int>> adjacency_;
std::vector<int> left_match_;
std::vector<int> right_match_;

```

```
std::vector<int> distance_;  
};
```

Checklist

- Modela aristas dirigidas en Dinic; para una arista no dirigida agrega ambas direcciones según la capacidad requerida.
- En Hopcroft-Karp, separa claramente los dos conjuntos de vértices antes de insertar aristas.
- Usa `long long` si el flujo total puede superar `int`.

Part III: Advanced topics

21 Number theory

21.1 Primes and factors

21.2 Modular arithmetic

21.3 Solving equations

21.4 Other results

Complemento UTFSM: teoría de números avanzada

Teorema chino del resto

Combina dos congruencias, incluso si los módulos no son coprimos. Devuelve (residuo, modulo) normalizado, o `std::nullopt` si no hay solución o el mínimo común múltiplo no cabe en `long long`. La complejidad es $O(\log \min(m, n))$.

Crédito UTFSM: Sebastián Torrealba contribuyó la implementación de referencia en el Handbook UTFSM.

```
#include <limits>
#include <optional>
#include <utility>

long long add_modulo(long long first, long long second, long long modulus) {
    if (modulus == 1) {
        return 0;
    }
    return first >= modulus - second ? first - (modulus - second) : first + second;
}

long long multiply_modulo(long long first, long long second, long long modulus) {
    long long result = 0;
    while (second > 0) {
        if (second & 1LL) {
            result = add_modulo(result, first, modulus);
        }
        first = add_modulo(first, first, modulus);
        second >>= 1LL;
    }
    return result;
}

long long extended_gcd(long long first, long long second, long long& coefficient_first,
                      long long& coefficient_second) {
    if (second == 0) {
        coefficient_first = first >= 0 ? 1 : -1;
        coefficient_second = 0;
        return first >= 0 ? first : -first;
    }
    long long next_first = 0;
    long long next_second = 0;
    const long long divisor = extended_gcd(second, first % second, next_first, next_second);
    coefficient_first = next_second;
    coefficient_second = next_first - (first / second) * next_second;
    return divisor;
}

std::optional<std::pair<long long, long long>> chinese_remainder(
    long long first_remainder, long long first_modulus, long long second_remainder,
    long long second_modulus) {
    if (first_modulus <= 0 || second_modulus <= 0) {
        return std::nullopt;
    }
    first_remainder = (first_remainder % first_modulus + first_modulus) % first_modulus;
```

```

second_remainder = (second_remainder % second_modulus + second_modulus) % second_modulus;
long long inverse = 0;
long long unused = 0;
const long long divisor =
    extended_gcd(first_modulus, second_modulus, inverse, unused);
const long long difference = second_remainder - first_remainder;
if (difference % divisor != 0) {
    return std::nullopt;
}
const long long reduced_modulus = second_modulus / divisor;
if (first_modulus / divisor >
    std::numeric_limits<long long>::max() / second_modulus) {
    return std::nullopt;
}
const long long lcm = first_modulus / divisor * second_modulus;
const long long normalized_difference =
    ((difference / divisor) % reduced_modulus + reduced_modulus) % reduced_modulus;
const long long normalized_inverse =
    (inverse % reduced_modulus + reduced_modulus) % reduced_modulus;
const long long multiplier =
    multiply_modulo(normalized_difference, normalized_inverse, reduced_modulus);
const long long increment = first_modulus * multiplier;
return std::pair<long long, long long>{add_modulo(first_remainder, increment, lcm),
    lcm};
}

```

Miller-Rabin determinista para 64 bits

`is_prime(number)` prueba primalidad exacta para todo `uint64_t` mediante una base de testigos válida en ese dominio. Usa multiplicación de 128 bits y tarda $O(\log \text{ number})$ por testigo.

Crédito UTFSM: Sebastián Torrealba contribuyó la implementación de referencia en el Handbook UTFSM.

```

#include <array>
#include <cstdint>

using Unsigned64 = std::uint64_t;
Unsigned64 multiply_modulo(Unsigned64 first, Unsigned64 second, Unsigned64 modulus) {
    Unsigned64 result = 0;
    while (second > 0) {
        if (second & 1U) {
            result = result >= modulus - first ? result - (modulus - first) : result + first;
        }
        first = first >= modulus - first ? first - (modulus - first) : first + first;
        second >>= 1U;
    }
    return result;
}

Unsigned64 power_modulo(Unsigned64 base, Unsigned64 exponent, Unsigned64 modulus) {
    Unsigned64 result = 1;
    while (exponent > 0) {
        if (exponent & 1U) {
            result = multiply_modulo(result, base, modulus);
        }
        base = multiply_modulo(base, base, modulus);
        exponent >>= 1U;
    }
    return result;
}

```

```

bool is_prime(Unsigned64 number) {
    if (number < 2) {
        return false;
    }
    for (const Unsigned64 prime : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (number % prime == 0) {
            return number == prime;
        }
    }
    Unsigned64 odd_part = number - 1;
    int exponent = 0;
    while ((odd_part & 1U) == 0) {
        odd_part >>= 1U;
        ++exponent;
    }
    constexpr std::array<Unsigned64, 7> witnesses{
        2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL};
    for (const Unsigned64 witness : witnesses) {
        if (witness % number == 0) {
            continue;
        }
        Unsigned64 value = power_modulo(witness % number, odd_part, number);
        if (value == 1 || value == number - 1) {
            continue;
        }
        bool reached_minus_one = false;
        for (int iteration = 1; iteration < exponent; ++iteration) {
            value = multiply_modulo(value, value, number);
            if (value == number - 1) {
                reached_minus_one = true;
                break;
            }
        }
        if (!reached_minus_one) {
            return false;
        }
    }
    return true;
}

```

Precauciones

- No conviertas números negativos a `uint64_t` antes de llamar a `is_prime`.
- El CRT admite módulos positivos solamente.
- Si necesitas combinar más de dos congruencias, combina el resultado con la siguiente de forma iterativa.

22 Combinatorics

22.1 Binomial coefficients

22.2 Catalan numbers

22.3 Inclusion-exclusion

22.4 Burnside's lemma

22.5 Cayley's formula

23 Matrices

23.1 Operations

23.2 Linear recurrences

23.3 Graphs and matrices

Complemento UTFSM: multiplicación rápida de polinomios

Number Theoretic Transform

Multiplica polinomios módulo 998244353 mediante NTT. `convolve(a, b)` funciona en $O(N \log N)$, donde N es la siguiente potencia de dos que cubre el grado del resultado. Sus coeficientes de entrada deben estar normalizados en $[0, 998244353)$.

Crédito UTFSM: Sebastián Torrealba contribuyó la implementación de referencia en el Handbook UTFSM.

```
#include <algorithm>
#include <vector>

class NumberTheoreticTransform {
public:
    static constexpr int modulus = 998244353;
    static constexpr int primitive_root = 3;

    std::vector<int> convolve(std::vector<int> first, std::vector<int> second) const {
        if (first.empty() || second.empty()) {
            return {};
        }
        const int result_size = static_cast<int>(first.size() + second.size() - 1);
        int transform_size = 1;
        while (transform_size < result_size) {
            transform_size *= 2;
        }
        first.resize(transform_size);
        second.resize(transform_size);
        transform(first, false);
        transform(second, false);
        for (int index = 0; index < transform_size; ++index) {
            first[index] = multiply(first[index], second[index]);
        }
        transform(first, true);
        first.resize(result_size);
        return first;
    }

private:
    static int multiply(long long first, long long second) {
        return static_cast<int>(first * second % modulus);
    }

    static int power(int base, int exponent) {
        int result = 1;
        while (exponent > 0) {
            if (exponent & 1) {
                result = multiply(result, base);
            }
            base = multiply(base, base);
            exponent >>= 1;
        }
        return result;
    }
}
```

```

static void transform(std::vector<int>& values, bool invert) {
    const int size = static_cast<int>(values.size());
    for (int index = 1, reversed = 0; index < size; ++index) {
        int bit = size >> 1;
        for (; reversed & bit; bit >>= 1) {
            reversed ^= bit;
        }
        reversed ^= bit;
        if (index < reversed) {
            std::swap(values[index], values[reversed]);
        }
    }
    for (int block_size = 2; block_size <= size; block_size *= 2) {
        int root = power(primitive_root, (modulus - 1) / block_size);
        if (invert) {
            root = power(root, modulus - 2);
        }
        for (int start = 0; start < size; start += block_size) {
            int current_root = 1;
            for (int offset = 0; offset < block_size / 2; ++offset) {
                const int first = values[start + offset];
                const int second = multiply(values[start + offset + block_size / 2], current_root);
                values[start + offset] = first + second < modulus ? first + second : first + second - modulus;
                values[start + offset + block_size / 2] = first - second >= 0 ? first - second : first - second + modulus;
                current_root = multiply(current_root, root);
            }
        }
    }
    if (invert) {
        const int inverse_size = power(size, modulus - 2);
        for (int& value : values) {
            value = multiply(value, inverse_size);
        }
    }
};

```

La raíz primitiva y el módulo están fijados porque el módulo permite longitudes potencia de dos hasta 2^{23} . Para otro módulo, usa CRT entre NTT compatibles o una FFT con control de redondeo.

24 Probability

24.1 Calculation

24.2 Events

24.3 Random variables

24.4 Markov chains

24.5 Randomized algorithms

25 Game theory

25.1 Game states

25.2 Nim game

25.3 Sprague–Grundy theorem

26 String algorithms

26.1 String terminology

26.2 Trie structure

26.3 String hashing

26.4 Z-algorithm

Complemento UTFSM: algoritmos avanzados de strings

KMP

`kmp_search(text, pattern)` devuelve todos los índices de inicio de coincidencias, incluso solapadas, en $O(|text| + |pattern|)$. El patrón vacío devuelve una lista vacía por decisión explícita de la interfaz.

Crédito UTFSM: Gabriel Carmona contribuyó la implementación de referencia en el Handbook UTFSM.

```
#include <string>
#include <vector>

std::vector<int> prefix_function(const std::string& text) {
    std::vector<int> prefix(text.size());
    for (int index = 1; index < static_cast<int>(text.size()); ++index) {
        int candidate = prefix[index - 1];
        while (candidate > 0 && text[index] != text[candidate]) {
            candidate = prefix[candidate - 1];
        }
        if (text[index] == text[candidate]) {
            ++candidate;
        }
        prefix[index] = candidate;
    }
    return prefix;
}

std::vector<int> kmp_search(const std::string& text, const std::string& pattern) {
    if (pattern.empty()) {
        return {};
    }
    const std::vector<int> prefix = prefix_function(pattern);
    std::vector<int> matches;
    int matched = 0;
    for (int index = 0; index < static_cast<int>(text.size()); ++index) {
        while (matched > 0 && text[index] != pattern[matched]) {
            matched = prefix[matched - 1];
        }
        if (text[index] == pattern[matched]) {
            ++matched;
        }
        if (matched == static_cast<int>(pattern.size())) {
            matches.push_back(index - matched + 1);
            matched = prefix[matched - 1];
        }
    }
    return matches;
}
```

Aho-Corasick

Busca múltiples patrones simultáneamente en tiempo lineal respecto al texto más las coincidencias. Esta implementación acepta solamente letras minúsculas a-z; agrega patrones, llama una vez a `build` y luego a `search`. Cada resultado contiene (`índice final`, `id_del_patron`).

Crédito UTFSM: Sebastián Torrealba contribuyó la implementación de referencia en el Handbook UTFSM.

```
#include <array>
#include <queue>
#include <string>
#include <utility>
#include <vector>

class AhoCorasick {
public:
    AhoCorasick() { nodes_.push_back(Node{}); }

    void add_pattern(const std::string& pattern, int pattern_id) {
        int node = 0;
        for (const char character : pattern) {
            const int letter = character - 'a';
            if (nodes_[node].next[letter] == -1) {
                nodes_[node].next[letter] = static_cast<int>(nodes_.size());
                nodes_.push_back(Node{});
            }
            node = nodes_[node].next[letter];
        }
        nodes_[node].output.push_back(pattern_id);
    }

    void build() {
        std::queue<int> pending;
        for (int letter = 0; letter < 26; ++letter) {
            const int child = nodes_[0].next[letter];
            if (child == -1) {
                nodes_[0].next[letter] = 0;
            } else {
                pending.push(child);
            }
        }
        while (!pending.empty()) {
            const int node = pending.front();
            pending.pop();
            for (int letter = 0; letter < 26; ++letter) {
                const int child = nodes_[node].next[letter];
                if (child == -1) {
                    nodes_[node].next[letter] = nodes_[nodes_[node].link].next[letter];
                    continue;
                }
                nodes_[child].link = nodes_[nodes_[node].link].next[letter];
                const auto& inherited_output = nodes_[nodes_[child].link].output;
                nodes_[child].output.insert(nodes_[child].output.end(),
                                           inherited_output.begin(), inherited_output.end());
                pending.push(child);
            }
        }
    }

    std::vector<std::pair<int, int>> search(const std::string& text) const {
        std::vector<std::pair<int, int>> matches;
        int node = 0;
        for (int index = 0; index < static_cast<int>(text.size()); ++index) {
            node = nodes_[node].next[text[index] - 'a'];
            for (const int pattern_id : nodes_[node].output) {
                matches.push_back({index, pattern_id});
            }
        }
    }
};
```

```

    }
    return matches;
}

private:
struct Node {
    std::array<int, 26> next{};
    int link = 0;
    std::vector<int> output;

    Node() { next.fill(-1); }
};

std::vector<Node> nodes_;
};

```

Suffix array y LCP

`suffix_array` ordena los sufijos en $O(N \log^2 N)$ y `lcp_array` calcula los LCP adyacentes en $O(N)$. Ambos aceptan cualquier byte de `std::string`; para texto vacío devuelven vectores vacíos.

Crédito UTFSM: Sebastián Torrealba contribuyó la implementación de referencia en el Handbook UTFSM.

```

#include <algorithm>
#include <numeric>
#include <string>
#include <vector>

std::vector<int> suffix_array(const std::string& text) {
    const int size = static_cast<int>(text.size());
    std::vector<int> order(size);
    std::iota(order.begin(), order.end(), 0);
    std::vector<int> rank(size);
    for (int index = 0; index < size; ++index) {
        rank[index] = static_cast<unsigned char>(text[index]);
    }
    for (int length = 1; length < size; length *= 2) {
        const auto compare = [&](int first, int second) {
            if (rank[first] != rank[second]) {
                return rank[first] < rank[second];
            }
            const int first_next = first + length < size ? rank[first + length] : -1;
            const int second_next = second + length < size ? rank[second + length] : -1;
            return first_next < second_next;
        };
        std::sort(order.begin(), order.end(), compare);
        std::vector<int> next_rank(size);
        for (int index = 1; index < size; ++index) {
            next_rank[order[index]] = next_rank[order[index - 1]] +
                (compare(order[index - 1], order[index]) ? 1 : 0);
        }
        rank.swap(next_rank);
        if (rank[order.back()] == size - 1) {
            break;
        }
    }
    return order;
}

std::vector<int> lcp_array(const std::string& text, const std::vector<int>& order) {
    std::vector<int> rank(text.size());

```

```

for (int index = 0; index < static_cast<int>(order.size()); ++index) {
    rank[order[index]] = index;
}
std::vector<int> lcp(text.size() > 0 ? text.size() - 1 : 0);
int common = 0;
for (int index = 0; index < static_cast<int>(text.size()); ++index) {
    if (rank[index] == static_cast<int>(text.size()) - 1) {
        common = 0;
        continue;
    }
    const int following = order[rank[index] + 1];
    while (index + common < static_cast<int>(text.size()) &&
           following + common < static_cast<int>(text.size()) &&
           text[index + common] == text[following + common]) {
        ++common;
    }
    lcp[rank[index]] = common;
    common = std::max(0, common - 1);
}
return lcp;
}

```

Manacher

Calcula radios de palíndromos pares e impares en $O(N)$. En `odd_palindrome_radii`, el radio incluye el centro; en `even_palindrome_radii`, indica cuántos caracteres se extienden a cada lado del corte.

Crédito UTFSM: Sebastián Torrealba contribuyó la implementación de referencia en el Handbook UTFSM.

```

#include <algorithm>
#include <string>
#include <vector>

std::vector<int> odd_palindrome_radii(const std::string& text) {
    const int size = static_cast<int>(text.size());
    std::vector<int> radius(size);
    int left = 0;
    int right = -1;
    for (int center = 0; center < size; ++center) {
        int current = center > right ? 1 : std::min(radius[left + right - center], right - center + 1);
        while (center - current >= 0 && center + current < size &&
               text[center - current] == text[center + current]) {
            ++current;
        }
        radius[center] = current;
        if (center + current - 1 > right) {
            left = center - current + 1;
            right = center + current - 1;
        }
    }
    return radius;
}

std::vector<int> even_palindrome_radii(const std::string& text) {
    const int size = static_cast<int>(text.size());
    std::vector<int> radius(size);
    int left = 0;
    int right = -1;
    for (int center = 0; center < size; ++center) {
        int current = center > right ? 0 : std::min(radius[left + right - center + 1], right - center + 1);
        while (center - current - 1 >= 0 && center + current < size &&

```

```
        text[center - current - 1] == text[center + current])) {
            ++current;
        }
        radius[center] = current;
        if (center + current - 1 > right) {
            left = center - current;
            right = center + current - 1;
        }
    }
    return radius;
}
```

27 Square root algorithms

27.1 Combining algorithms

27.2 Integer partitions

27.3 Mo's algorithm

28 Segment trees revisited

28.1 Lazy propagation

28.2 Dynamic trees

28.3 Data structures

28.4 Two-dimensionality

29 Geometry

29.1 Complex numbers

29.2 Points and lines

29.3 Polygon area

29.4 Distance functions

Complemento UTFSM: geometría computacional avanzada

Convex hull

Construye el casco convexo de puntos enteros en sentido antihorario mediante monotonic chain. Tarda $O(N \log N)$, no repite el primer punto al final y elimina puntos colineales interiores.

Crédito UTFSM: Sebastián Torrealba contribuyó la implementación de referencia en el Handbook UTFSM.

```
#include <algorithm>
#include <vector>

struct IntegerPoint {
    long long x;
    long long y;

    bool operator<(const IntegerPoint& other) const {
        return x != other.x ? x < other.x : y < other.y;
    }

    bool operator==(const IntegerPoint& other) const {
        return x == other.x && y == other.y;
    }
};

long long cross(const IntegerPoint& origin, const IntegerPoint& first,
               const IntegerPoint& second) {
    return (first.x - origin.x) * (second.y - origin.y) -
           (first.y - origin.y) * (second.x - origin.x);
}

std::vector<IntegerPoint> convex_hull(std::vector<IntegerPoint> points) {
    std::sort(points.begin(), points.end());
    points.erase(std::unique(points.begin(), points.end()), points.end());
    if (points.size() <= 2) {
        return points;
    }
    std::vector<IntegerPoint> hull;
    for (const IntegerPoint& point : points) {
        while (hull.size() >= 2 &&
              cross(hull[hull.size() - 2], hull.back(), point) <= 0) {
            hull.pop_back();
        }
        hull.push_back(point);
    }
    const std::size_t lower_size = hull.size();
    for (int index = static_cast<int>(points.size()) - 2; index >= 0; --index) {
        while (hull.size() > lower_size &&
              cross(hull[hull.size() - 2], hull.back(), points[index]) <= 0) {
            hull.pop_back();
        }
        hull.push_back(points[index]);
    }
    hull.pop_back();
    return hull;
}
```

```
}
```

Punto en polígono

Clasifica un punto como interior, borde o exterior con ray casting. Funciona para polígonos simples y usa `long double`; la respuesta de borde depende del epsilon documentado.

Crédito UTFSM: Sebastián Torrealba contribuyó la implementación de referencia en el Handbook UTFSM.

```
#include <cmath>
#include <vector>

struct RealPoint {
    long double x;
    long double y;
};

enum class PointLocation { kOutside, kBoundary, kInside };

long double cross(const RealPoint& first, const RealPoint& second,
                  const RealPoint& third) {
    return (second.x - first.x) * (third.y - first.y) -
           (second.y - first.y) * (third.x - first.x);
}

bool on_segment(const RealPoint& first, const RealPoint& second,
                const RealPoint& point, long double epsilon = 1e-12L) {
    return std::fabs(cross(first, second, point)) <= epsilon &&
           point.x >= std::min(first.x, second.x) - epsilon &&
           point.x <= std::max(first.x, second.x) + epsilon &&
           point.y >= std::min(first.y, second.y) - epsilon &&
           point.y <= std::max(first.y, second.y) + epsilon;
}

PointLocation point_in_polygon(const std::vector<RealPoint>& polygon,
                              const RealPoint& point) {
    bool inside = false;
    for (int current = 0, previous = static_cast<int>(polygon.size()) - 1;
         current < static_cast<int>(polygon.size()); previous = current++) {
        const RealPoint& first = polygon[previous];
        const RealPoint& second = polygon[current];
        if (on_segment(first, second, point)) {
            return PointLocation::kBoundary;
        }
        const bool crosses_horizontal_ray = (first.y > point.y) != (second.y > point.y);
        if (crosses_horizontal_ray &&
            point.x < (second.x - first.x) * (point.y - first.y) /
                    (second.y - first.y) + first.x) {
            inside = !inside;
        }
    }
    return inside ? PointLocation::kInside : PointLocation::kOutside;
}
```

Intersección de semiplanos

Interseca semiplanos orientados: el interior de cada uno debe quedar a la izquierda de su dirección. Corre en $O(N \log N)$ y devuelve el polígono convexo de intersección, o un vector vacío si la intersección es vacía o no queda acotada.

Crédito UTFSM: Sebastián Torrealba contribuyó la implementación de referencia en el Handbook UTFSM.

```

#include <algorithm>
#include <cmath>
#include <deque>
#include <vector>

struct HalfPlanePoint {
    long double x;
    long double y;

    HalfPlanePoint operator+(const HalfPlanePoint& other) const {
        return {x + other.x, y + other.y};
    }

    HalfPlanePoint operator-(const HalfPlanePoint& other) const {
        return {x - other.x, y - other.y};
    }

    HalfPlanePoint operator*(long double factor) const { return {x * factor, y * factor}; }
};

long double cross(const HalfPlanePoint& first, const HalfPlanePoint& second) {
    return first.x * second.y - first.y * second.x;
}

struct HalfPlane {
    HalfPlanePoint point;
    HalfPlanePoint direction;

    long double angle() const { return std::atan2(direction.y, direction.x); }

    bool excludes(const HalfPlanePoint& candidate, long double epsilon = 1e-12L) const {
        return cross(direction, candidate - point) < -epsilon;
    }
};

HalfPlanePoint intersection(const HalfPlane& first, const HalfPlane& second) {
    const long double scale =
        cross(second.point - first.point, second.direction) /
        cross(first.direction, second.direction);
    return first.point + first.direction * scale;
}

std::vector<HalfPlanePoint> half_plane_intersection(std::vector<HalfPlane> planes) {
    std::sort(planes.begin(), planes.end(), [](const HalfPlane& first,
        const HalfPlane& second) {
        return first.angle() < second.angle();
    });
    std::deque<HalfPlane> active;
    for (const HalfPlane& plane : planes) {
        while (active.size() > 1 &&
            plane.excludes(intersection(active[active.size() - 2], active.back()))) {
            active.pop_back();
        }
        while (active.size() > 1 && plane.excludes(intersection(active[0], active[1]))) {
            active.pop_front();
        }
        if (!active.empty() && std::fabs(cross(active.back().direction, plane.direction)) <= 1e-12L) {
            if (plane.excludes(active.back().point)) {
                active.back() = plane;
            }
            continue;
        }
    }
}

```

```

    }
    active.push_back(plane);
}
while (active.size() > 2 &&
       active.front().excludes(intersection(active[active.size() - 2], active.back()))) {
    active.pop_back();
}
while (active.size() > 2 &&
       active.back().excludes(intersection(active[0], active[1]))) {
    active.pop_front();
}
if (active.size() < 3) {
    return {};
}
std::vector<HalfPlanePoint> polygon;
for (int index = 0; index < static_cast<int>(active.size()); ++index) {
    polygon.push_back(intersection(active[index], active[(index + 1) % active.size()]));
}
return polygon;
}

```

Checklist numérico

- Mantén una convención única de orientación en todas las primitivas.
- Evita mezclar `double` y enteros cuando el problema requiera exactitud.
- Prueba casos degenerados: puntos colineales, aristas horizontales y semiplanos paralelos.

30 Sweep line algorithms

30.1 Intersection points

30.2 Closest pair problem

30.3 Convex hull problem

Bibliography